

Desarrolla tu App Flutter **sin saber nada de Flutter**

La Era Agéntica y el uso de Coding Agents

ACERCA DEL PONENTE

Juan Guillermo Gómez



Roles & Reconocimiento

- Co-leader and co-founder of GDG Cali.
- AI & Software Advisor & Freelancer in DevHack.
- Google Developer Expert (GDE) in Firebase & GCP & AI/ML



Formación Académica

- BS in System Engineering
- M.Sc in Software Engineering
- M.Sc AI & Data Science

@jggomezt

 devhack.co



 Experts

Networking



LinkedIn



X



Instagram



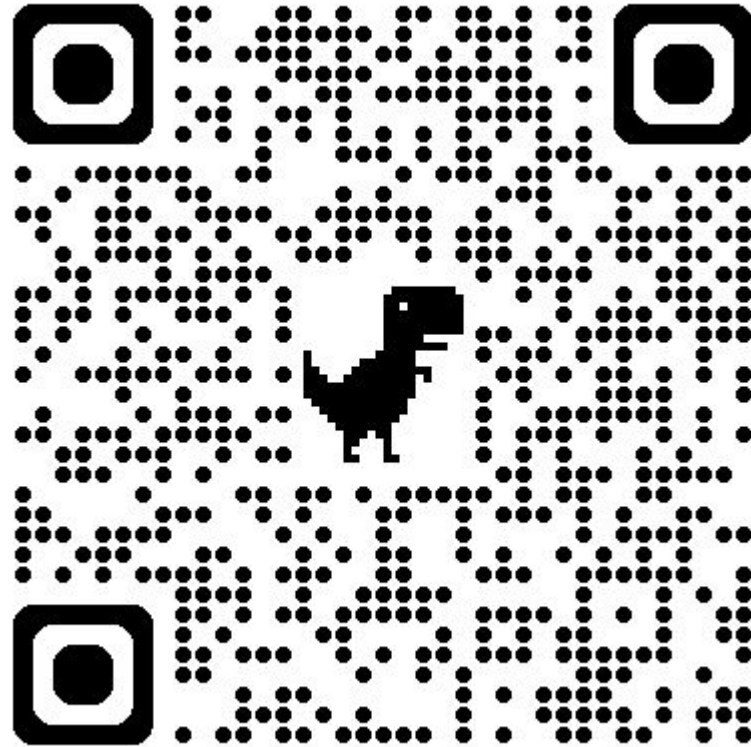
Onlyfans

Networking

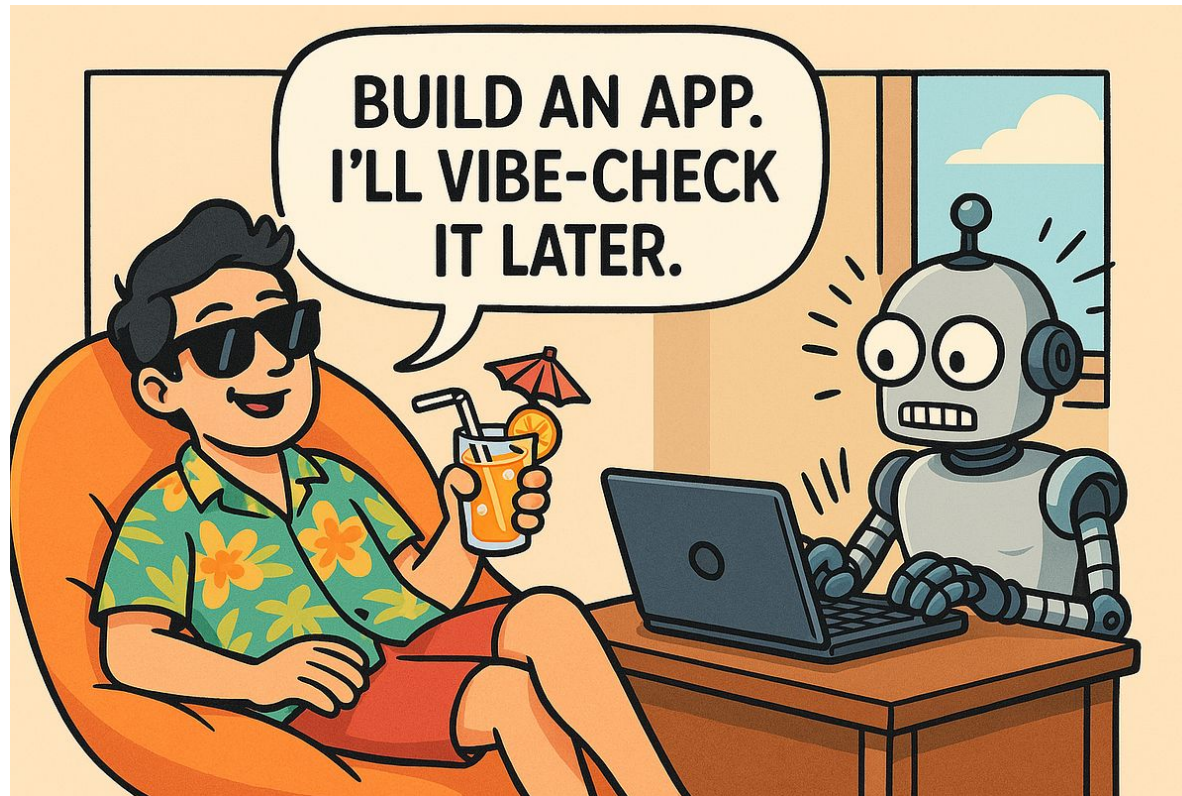


Youtube

Quieres Participar por Premios



Historia... porque estoy aquí



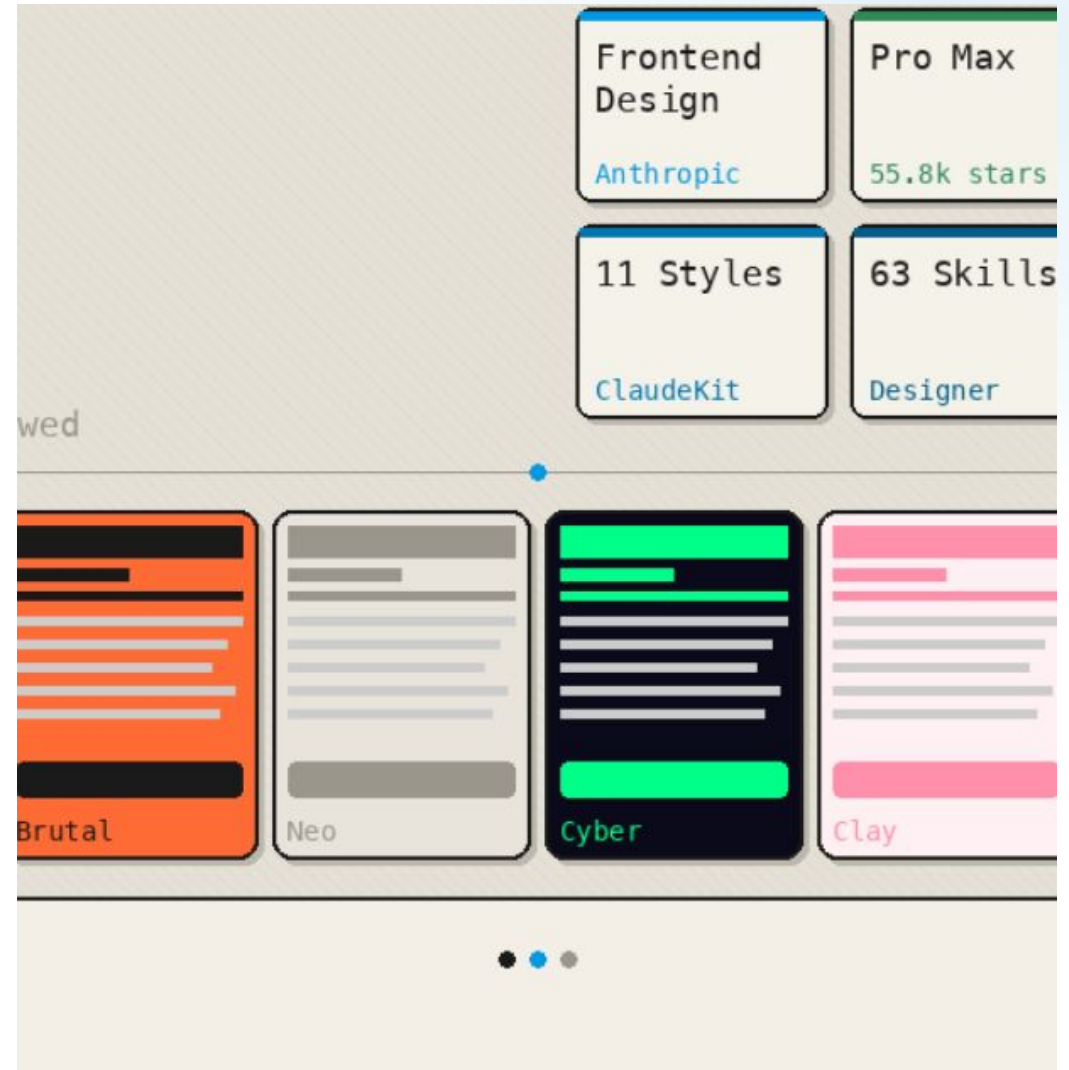
El Espejismo de la Velocidad

La Ilusión de la Rapidez

Generar código con IA es **instantáneo**, pero el razonamiento y la auditoría humana se mantienen en un **ancho de banda constante**.

El Riesgo del Falso Rigor

Aceptar cambios solo porque compilan **no es programar con rigor**: es acumular **deuda de comprensión** y trasladar el riesgo directo al usuario final.



Las Tres Deudas Invisibles



01

Comprehension Debt

La brecha entre el código acumulado y lo que el equipo entiende.

El software compila, pero nadie comprende el sistema desde primeros principios si ocurre una falla.



02

Intent Debt

Pérdida del porqué del diseño.

Si no existe especificación explícita, la IA fabrica explicaciones falsas para rellenar vacíos lógicos del negocio.



03

Cognitive Surrender

Rendición pasiva ante el modelo:

Aceptar ciegamente la salida del agente adoptando su alta confianza probabilística y atrofiando el juicio crítico.

La Tasa de Orquestación y el GIL



La Tasa de Orquestación

Lanzar diez agentes concurrentes no multiplica la velocidad por diez. Crear tareas es barato; verificar, validar contratos e integrar ramas concurrentes es sumamente costoso.

La saturación aparece cuando el tiempo de coordinar supera al tiempo que tomaría implementar directamente la solución en Flutter.



El Cerebro como GIL

El juicio crítico del Tech Lead funciona igual que el **Global Interpreter Lock**: es un recurso monohilo, finito y strictly no paralelizable.

Cada cambio de agente impone una recarga en frío del modelo mental, desembocando en fatiga cognitiva y aprobaciones superficiales.

MÓDULO 2



Infraestructura de Control

Harness Engineering y Context Engineering: los cimientos que transforman modelos probabilísticos en sistemas de producción gobernables.

La Ecuación Agéntica y el Arnés

Coding Agent = AI Model + **Harness (Arnés)**



Guides (Feedforward)

Orientan al agente **antes** de generar código:

- Especificaciones en Markdown
- Contratos de Clean Architecture
- Tipado estricto
- Convenciones de Riverpod



Sensors (Feedback)

Evalúan al agente **después** de escribir:

- Linters deterministas (dart analyze)
- Pruebas unitarias
- Analizadores que fuerzan la autocorrección

El Trinquete (Matraca) Técnico



El Principio del Trinquete (Ratchet)

Ante cada fallo o alucinación del agente en Flutter, se programa una regla determinista para que ese error sea físicamente imposible de repetir.



Reglas Pasivas en AGENTS.md

Documento vivo curado por humanos. Cada directriz responde a un incidente real del pasado y no a prosa decorativa.



Tablas Anti-Racionalización

Instrucciones estrictas en las skills que desmontan las excusas del modelo antes de que las use (ej. prohibido omitir tests alegando que se harán después).



Verificación como Condición de Salida

Ninguna meta se da por concluida hasta que el sensor determinista emita un reporte 100% verde sin advertencias.

Ingeniería de Loops Autónomos



Loops Recursivos

Pasamos de interactuar con prompts turno a turno a diseñar loops recursivos en lazo cerrado que convergen hacia una meta formal.

- **Lazo Cerrado:** Convergencia continua hacia metas formalmente definidas.
- **Autonomía:** Supera la interacción iterativa prompt a prompt.
- **Ejecución Recursiva:** Ciclos continuos de mejora y refinamiento.



Arquitectura Maker / Checker

La arquitectura Maker/Checker separa al agente que genera código del sensor que lo audita, eliminando autoevaluaciones indulgentes.

- **Maker (Generador):** Agente enfocado puramente en crear la solución y código.
- **Checker (Auditor):** Sensor independiente que valida objetivamente el resultado.
- **Objetividad:** Elimina sesgos y autoevaluaciones complacientes.

Fábricas de Software: Lit vs Dark

Dimensión	Dark Factory (Fábrica Oscura)	Lit Factory (Fábrica Iluminada)
Control Humano	Totalmente delegado; nadie revisa los diffs	Prioridad en especificación y arquitectura
Sensor Requerido	Oráculo infalsificable y determinista	Pruebas de arquitectura + verificación deliberada
Riesgo Principal	Deuda de comprensión masiva e inmanejable	Mayor inversión inicial al formalizar la spec
Ámbito de Uso	Scripts descartables o tareas reversibles	Sistemas empresariales y apps Flutter en producción

Dominar el Bucle Externo



Bucle Interno (La IA)

La IA es la dueña del **cómo sintáctico**: escribir boilerplate, ensamblar widgets de Flutter, ejecutar comandos de terminal y refactorizar llamadas de estado.

Opera como un motor probabilístico incansable que requiere supervisión constante en sus límites de ejecución.

- **Ejecución sintáctica**: Generación veloz de código boilerplate.
- **Probabilístico**: Respuestas basadas en patrones estadísticos.
- **Supervisión necesaria**: Límites en razonamiento abstracto.

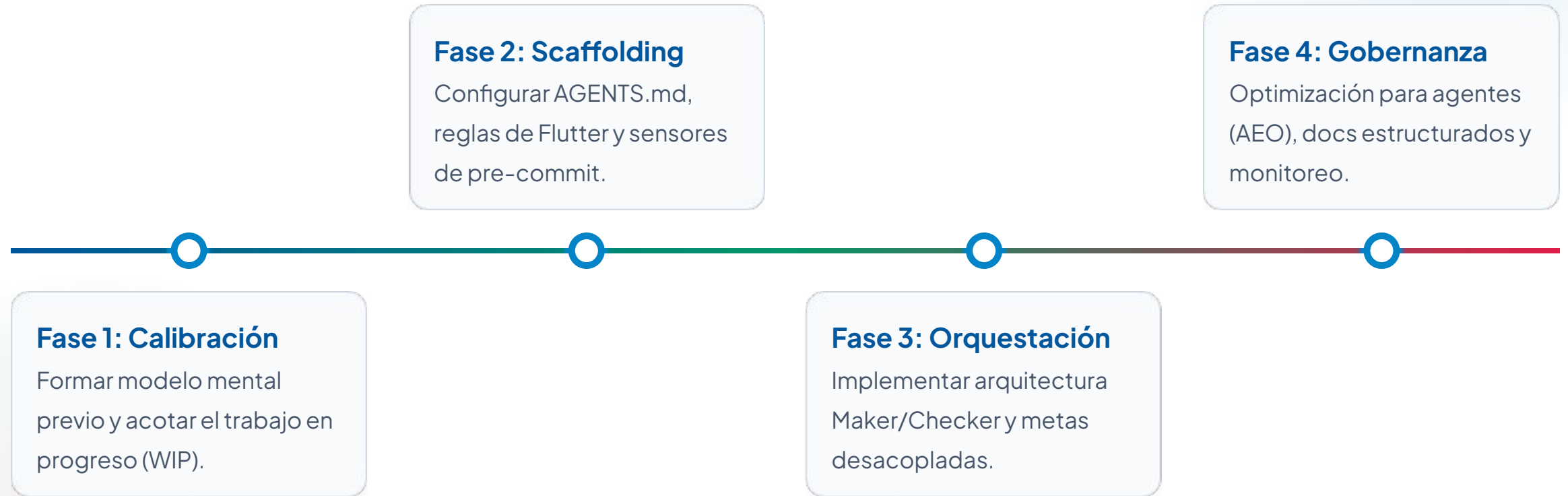


Bucle Externo (El Ingeniero)

El humano es el dueño del **qué y el porqué** arquitectónico mediante tres facultades irremplazables:

- **1. Evidencia**: Exigir registros y diffs verificables.
- **2. Veredicto**: Decidir qué código entra a producción.
- **3. Responsabilidad**: Responder por el sistema ante los usuarios.

Ruta a la Autonomía Calibrada



Mapa de Habilidades en Ingeniería de IA (Andrew Ng)

Las 4 competencias clave para liderar el desarrollo en la era de los agentes



1. Construcción y Despliegue

- **Desafío:** Manejar la impredecibilidad del modelo frente al SW tradicional.
- **Dominio:** Integrar LLMs, RAG, context engineering y flujos agénticos.
- **Gobernanza:** Métricas estadísticas, evals y análisis de errores.



2. Fundamentos de Ingeniería de Software

- **Gestión de Tradeoffs:** Balancear costo, escalabilidad, velocidad, fiabilidad y seguridad.
- **Lenguaje Técnico Preciso:** Evitar el "vibe coding" ciego, guiando al agente con vocabulario técnico estructurado.



3. Uso de Agentes de Programación

- **Modelo Mental Agéntico:** Comprender limitaciones, gestionar contexto y equilibrar planificación/ejecución.
- **Cierre de Loops:** Verificadores automatizados, specs claras y orquestación multi-agente.



4. Shaping the Build

- **Del "Cómo" al "Qué":** Con agentes resolviendo rápido, el valor pasa a definir qué construir.
- **Criterio de Producto:** Visión de negocio y cliente para decidir cuando lanzar un MVP o construir con cautela.

Referencias Clave en Ingeniería de IA

Investigaciones y conceptos fundamentales sobre agentes, gobernanza y el impacto en el razonamiento humano

Comprehension Debt

Osmani, A. (2026)

La brecha creciente entre la cantidad de código acumulado y la comprensión real que el equipo mantiene del sistema.

Agent Harness Engineering

Osmani, A. (2026)

La definición de Agente = Modelo + Harness y el impacto del soporte de ejecución en el rendimiento.

Loop Engineering

Osmani, A. (2026)

Transición del prompting interactivo al diseño de bucles recursivos autorregulados con memoria en disco.

Own the Outer Loop

Osmani, A. (2026)

Separación entre la ejecución del agente (bucle interno) y la responsabilidad técnica del desarrollador (bucle externo).

The Orchestration Tax

Osmani, A. (2026)

El cuello de botella monohilo de la atención humana al supervisar múltiples agentes en paralelo.

Software Factories: Light & Dark

Osmani, A. (2026)

Automatización a ciegas (fábrica oscura) vs. supervisión estratégica de decisiones de arquitectura (fábrica iluminada).

Cognitive Surrender

Osmani, A. (2026)

El peligro de aceptar ciegamente los resultados de la IA sin formar un juicio técnico previo.

AI Engineering Skills Map

Ng, A. (2026)

4 competencias clave: aplicaciones, fundamentos de ingeniería, agentes y criterio de producto.

Thinking — Fast, Slow & Artificial

Shaw, S. & Nave, G. (2026)

Estudio empírico sobre el aumento de confianza y la aceptación de respuestas incorrectas.

AI Assistance & Skill Formation

Anthropic Research (2026)

Reducción en la retención conceptual y habilidades de depuración tras la delegación pasiva.

 TALLER PRÁCTICO EN VIVO

Laboratorio: Cancún

DashBooth



App Web & Stack Tecnológico

Construiremos en vivo una aplicación web en Flutter para FlutterConf LATAM Cancún con Riverpod y Firebase AI Logic.

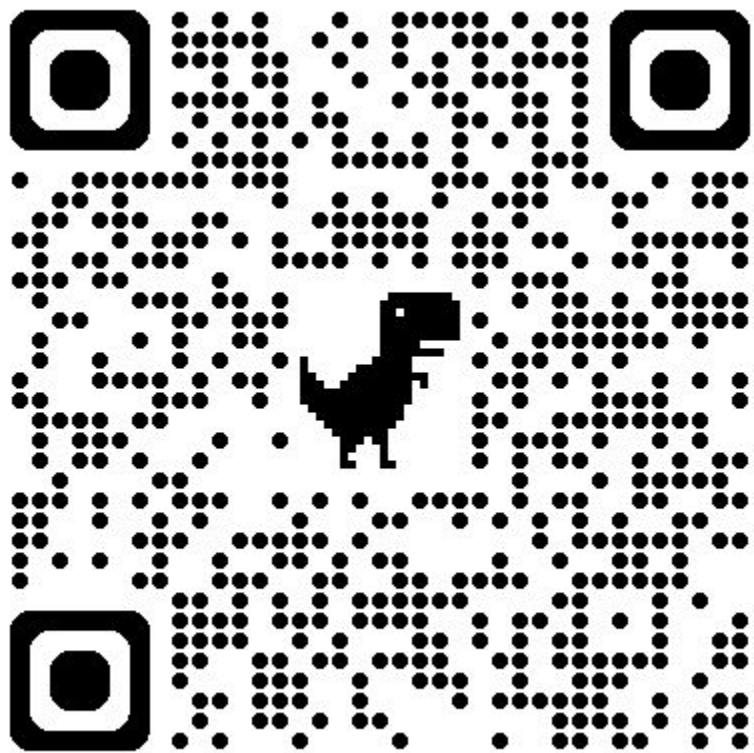


Gobernanza & Verificación Continua

Sin vibe coding ni improvisación: gobernados por especificaciones en /docs y Antigravity CLI con verificación determinista continua.



Repositorio Lab



¡MUCHAS GRACIAS!

¡Sigamos conectados!

Construyendo el futuro de la ingeniería con IA



LinkedIn



Contacto

juan.gomez01@gmail.com



Juan Guillermo Gómez Torres

Speaker & Tech Lead



Networking



LinkedIn



X



Instagram



Onlyfans